ASSIGNMENT-4

4.1) Consider the following table: Table: CLASS

ID	Name
1	Bravo
2	Alex
4	Cheng

Give the output of the following SQL script:

- INSERT INTO class VALUES (5,'Rahul');
- COMMIT;
- UPDATE class SET name = 'Abhijeet' WHERE id= '5';
- SAVEPOINT A;
- INSERT INTO class VALUES (6, 'Chris');
- SAVEPOINT B;
- INSERT INTO class VALUES (7, 'Bravo');
- SAVEPOINT C
- SELECT * FROM class;
- ROLLBACK TO B;
- SELECT * FROM class;
- ROLLBACK TO A;

SQL QUERY-

```
USE lab_db;
SET SQL_SAFE_UPDATES = 0;
DROP TABLE IF EXISTS class;

CREATE TABLE class (
   id INT,
   name VARCHAR(20)
);

INSERT INTO class VALUES
(1,'Bravo'),
(2,'Alex'),
(4,'Cheng');

– First transaction
START TRANSACTION;
INSERT INTO class VALUES (5,'Rahul');
COMMIT;
 – Ends transaction here

– Start NEW transaction
START TRANSACTION;
UPDATE class SET name = 'Abhijeet' WHERE id = 5;
SAVEPOINT A;
```

```
INSERT INTO class VALUES (6,'Chris');
SAVEPOINT B;

INSERT INTO class VALUES (7,'Bravo');
SAVEPOINT C;

– SELECT 1
SELECT * FROM class;
ROLLBACK TO B;

– SELECT 2
SELECT * FROM class;
ROLLBACK TO A;

– SELECT 3
SELECT * FROM class;
COMMIT;
```

OUTPUT-

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	name
1	Bravo
2	Alex
4	Cheng
5	Abhijeet

4.2) Consider the following table: Table: ACCOUNT_TXN

Account_ id	Holder _Name	Balance
101	Aman	5000
102	Bhavna	7000
103	Chirag	6000

Perform the following TCL-based SQL script and write the output after each SELECT statement:

```
INSERT INTO ACCOUNT_TXN VALUES (104, 'Diya', 4000);

SAVEPOINT S1;

UPDATE ACCOUNT_TXN SET Balance = Balance + 1500 WHERE Acc_ID = 101;

UPDATE ACCOUNT_TXN SET Balance = Balance - 1000 WHERE Acc_ID = 102;

SAVEPOINT S2;

DELETE FROM ACCOUNT_TXN WHERE Acc_ID = 103;

INSERT INTO ACCOUNT_TXN VALUES (105, 'Eshan', 3000);

SAVEPOINT S3;

UPDATE ACCOUNT_TXN SET Holder_Name = 'Bhavna Sharma' WHERE Acc_ID =
102;

SELECT * FROM ACCOUNT_TXN;

ROLLBACK TO S2;

SELECT * FROM ACCOUNT_TXN;

INSERT INTO ACCOUNT_TXN VALUES (106, 'Farah', 4500);

SAVEPOINT S4;

UPDATE ACCOUNT_TXN SET Balance = Balance + 500 WHERE Acc_ID IN (101,104);

SELECT * FROM ACCOUNT_TXN;

ROLLBACK TO S4;

SELECT * FROM ACCOUNT_TXN;

COMMIT;
```

Student Tasks

Write the output of all 4 SELECT statements in tabular format.

Identify which rows are permanently saved after COMMIT.

Mention which operations were undone by:

ROLLBACK TO S2

ROLLBACK TO S4

Write the final state of the table after COMMIT.

Additional Challenge (Conceptual)

SQL QUERY-

```
USE lab_db;
SET SQL_SAFE_UPDATES = 0;
DROP TABLE IF EXISTS ACCOUNT_TXN;

CREATE TABLE ACCOUNT_TXN (
   Acc_ID INT,
   Holder_Name VARCHAR(50),
   Balance INT
);
INSERT INTO ACCOUNT_TXN VALUES
(101,'Aman',5000),
(102,'Bhavna',7000),
(103,'Chirag',6000);
START TRANSACTION:
INSERT INTO ACCOUNT_TXN VALUES (104,'Diya',4000);
SAVEPOINT S1;
UPDATE ACCOUNT_TXN SET Balance = Balance + 1500 WHERE Acc_ID = 101;
UPDATE ACCOUNT_TXN SET Balance = Balance - 1000 WHERE Acc_ID = 102;
```

```
SAVEPOINT S2;
DELETE FROM ACCOUNT_TXN WHERE Acc_ID = 103;
INSERT INTO ACCOUNT_TXN VALUES (105,'Eshan',3000);

SAVEPOINT S3;
UPDATE ACCOUNT_TXN SET Holder_Name = 'Bhavna Sharma' WHERE Acc_ID =
102;
-- SELECT 1
SELECT * FROM ACCOUNT_TXN;
ROLLBACK TO S2;

-- SELECT 2
SELECT * FROM ACCOUNT_TXN;
INSERT INTO ACCOUNT_TXN VALUES (106,'Farah',4500);
SAVEPOINT S4;
UPDATE ACCOUNT_TXN
SET Balance = Balance + 500
WHERE Acc_ID IN (101,104);

-- SELECT 3
SELECT * FROM ACCOUNT_TXN;
ROLLBACK TO S4;


– SELECT 4
SELECT * FROM ACCOUNT_TXN;
COMMIT;
```

OUTPUT-

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Acc_ID	Holder_Name	Balance
101	Aman	6500
102	Bhavna	6000
103	Chirag	6000
104	Diya	4000
106	Farah	4500

Answer the following:

1)What is the difference between ROLLBACK and ROLLBACK TO SAVEPOINT?

ANS-

Feature	ROLLBACK	ROLLBACK TO SAVEPOINT
Meaning	Undoes the entire transaction	Undoes part of the transaction
Scope	Full rollback (all changes)	Partial rollback (up to a savepoint)
Savepoint required	❌ No	✅ Yes
Control	No control (all or nothing)	More control (step-by-step undo)
After execution	Transaction ends	Transaction continues

ROLLBACK undoes the entire transaction, whereas ROLLBACK TO SAVEPOINT undoes only up to a specific savepoint without ending the transaction.

2)Can we rollback to S3 after ROLLBACK TO S2? Why/Why not?

ANS-

No, we cannot rollback to S3 after executing ROLLBACK TO S2.

This is because when a rollback is performed to a particular savepoint (S2), all the savepoints created after that point (including S3) are automatically deleted by the system. As a result, savepoint S3 no longer exists in the transaction.

Therefore, it is not possible to rollback to S3 after rolling back to S2.

3)What happens if COMMIT is executed before creating any savepoint?

ANS-

If COMMIT is executed before creating any savepoint, all the changes made in the transaction are permanently saved in the database.

After the COMMIT:

- The transaction is terminated.
- No savepoints exist because none were created.
- It is not possible to perform a rollback to any previous state.

Thus, once COMMIT is executed, the changes cannot be undone, and a new transaction must be started for further operations.